Below is **ONE SINGLE MASTER TABLE** that covers **your entire repository**, aligned with **real-world projects, enterprise applications, and interview usage**.
This is **exam-ready, interview-ready, and project-mapping ready**.

✅ MASTER TABLE – JAVA DESIGN PATTERNS, CORE CONCEPTS & ENTERPRISE USAGE

No.	Category	Pattern / Concept	Core Purpose	Real-World Project / Application Usage
1	**Creational**	Singleton	Single instance	Logger, configuration, cache manager
2	**Creational**	Factory Method	Object creation abstraction	Object creation based on input
3	**Creational**	Abstract Factory	Factory of factories	Cloud providers, UI themes
4	**Creational**	Builder	Stepwise object creation	Complex objects (Order, User)
5	**Creational**	Prototype	Clone objects	Caching, performance optimization
6	**Structural**	Adapter	Interface conversion	Legacy system integration
7	**Structural**	Bridge	Abstraction decoupling	UI & backend separation
8	**Structural**	Composite	Tree structure	File systems, menus
9	**Structural**	Decorator	Add behavior dynamically	Logging, encryption, compression
10	**Structural**	Facade	Simplified interface	API gateways, service wrappers
11	**Structural**	Flyweight	Reduce memory usage	Caching, string pools
12	**Structural**	Proxy	Controlled access	Security, lazy loading
13	**Behavioral**	Chain of Responsibility	Pass request through handlers	Logging frameworks, approval workflows, request validation

14	**Behavioral**	Command	Encapsulate request as object	Undo/Redo, task queues, menu actions
15	**Behavioral**	Interpreter	Grammar evaluation	Expression engines, rule engines
16	**Behavioral**	Iterator	Sequential access	Collection traversal, DB cursors
17	**Behavioral**	Mediator	Centralized communication	Chat apps, UI coordination
18	**Behavioral**	Memento	Save/restore state	Undo feature, checkpoints
19	**Behavioral**	Observer	Event notification	Event listeners, pub-sub systems
20	**Behavioral**	State	State-based behavior	Order processing, workflow engines
21	**Behavioral**	Strategy	Switch algorithms	Payment gateways, sorting logic
22	**Behavioral**	Template Method	Algorithm skeleton	File parsers, report generation
23	**Behavioral**	Visitor	Add operations externally	Compilers, tax/report calculation
24	**J2EE – Presentation Tier**	Front Controller	Central request handling	Spring MVC DispatcherServlet
25	**J2EE – Presentation**	View Helper	JSP logic separation	JSP/Thymeleaf helpers
26	**J2EE – Presentation**	Composite View	Layout reuse	Tiles, UI templates
27	**J2EE – Presentation**	Dispatcher View	Navigation control	MVC routing
28	**J2EE – Presentation**	Intercepting Filter	Pre/post processing	Auth, logging filters
29	**J2EE – Presentation**	Application Controller	Workflow control	Multi-step processes
30	**J2EE – Business Tier**	Business Delegate	Hide service complexity	Client-service decoupling
31	**J2EE – Business**	Session Facade	Simplify business services	Service layer

32	**J2EE – Business**	Service Locator	Lookup services	JNDI, microservices
33	**J2EE – Business**	Transfer Object (DTO)	Data transport	REST request/response
34	**J2EE – Business**	Domain Model	Business entities	JPA entities
35	**J2EE – Integration Tier**	DAO	DB abstraction	JDBC/JPA access
36	**J2EE – Integration**	Service Activator	Async processing	Background jobs
37	**J2EE – Integration**	Web Service Broker	Service communicat ion	REST / SOAP APIs
38	**J2EE – Integration**	Messaging Gateway	Message abstraction	JMS, Kafka
39	**J2EE – Integration**	Business Object	Encapsulate logic	Core business services
40	**J2EE – Persistence Tier**	Data Mapper	ORM mapping	Hibernate
41	**J2EE – Persistence**	Identity Map	Cache objects	Reduce DB hits
42	**J2EE – Persistence**	Unit of Work	Transaction mgmt	JPA transactions
43	**J2EE – Persistence**	Lazy Loading	Load on demand	Performance tuning
44	**J2EE – Persistence**	Repository	CRUD abstraction	Spring Data
45	**J2EE – Persistence**	Active Record	Simple ORM	Lightweight CRUD
46	**J2EE – Persistence**	Query Object	Query abstraction	Criteria API
47	**Distributed & Concurrency**	Async Method Invocation	Non-blocking calls	Async services
48	**Distributed**	Thread Pool	Resource control	Executors

49	**Distributed**	Message Queue	Decoupled communicat ion	JMS, Kafka
50	**Distributed**	Object Pool	Reusable resources	DB connection pool
51	**Distributed**	Two Phase Commit	Distributed transactions	Banking systems
52	**Security**	RBAC	Role-based access	Enterprise authorization
53	**Security**	Secure Session	Session protection	Web security
54	**Security**	Authorization Interceptor	Access enforcement	API security
55	**Core Java**	Enums	Type-safe constants	Status codes
56	**Core Java**	SOLID Principles	Clean architecture	Maintainable code
57	**Core Java**	Annotations	Metadata	Spring, Hibernate
58	**Core Java**	Reflection	Runtime inspection	Framework internals
59	**Dependency Injection**	DI Pattern	Loose coupling	Spring Core
60	**MVC**	Model-View-Controller	UI separation	Spring MVC apps
61	**Servlet**	Servlet API	Web handling	Legacy & core web apps
62	**UML & OOP**	Association	Uses relationship	Object interaction
63	**UML & OOP**	Aggregation	Weak has-a	Department-Employee
64	**UML & OOP**	Composition	Strong has-a	Engine-Car
65	**UML & OOP**	Inheritance	Is-a	Class hierarchy
66	**UML & OOP**	Interface	Contract	Service APIs
67	**Integration & Auth**	LDAP Authentication	Central auth	Enterprise login

68	**Integration**	JAAS	Security framework	Java security

□ HOW TO USE THIS MASTER TABLE

- **✔Java interviews (Core → Advanced)**
- **✔System design discussions**
- **✔Spring / Microservices architecture**
- **✔Enterprise project planning**
- **✔Teaching & documentation**

Perfect question □
This is exactly how **senior developers, architects, and tech leads** think.

Below is a **REAL-PROJECT–ORIENTED mapping** — not theory — showing **which patterns are actually used**, **where**, and **why**.

✅ WHICH DESIGN PATTERNS TO USE IN REAL PROJECTS (PRACTICAL GUIDE)

1 Almost ALWAYS Used (Daily in Real Projects)

Area	Pattern	Where You Actually Use It
Object creation	**Singleton**	Config manager, Logger, Cache
Loose coupling	**Dependency Injection**	Spring Beans
Data access	**DAO / Repository**	Database layer
API structure	**DTO**	REST request/response
Web architecture	**MVC**	Spring MVC / REST
Simplification	**Facade**	Service layer wrappers
Security	**Proxy**	Security, access control
Events	**Observer**	Event listeners, messaging

Algorithms	**Strategy**	Payment, pricing, sorting

 If you don't use these, your project is already weak.

---

2 Very Common in Enterprise & Microservices

Use Case	Pattern	Real Example
Object creation logic	Factory Method	Payment method creation
Multiple variants	Abstract Factory	Cloud providers (AWS/Azure)
Workflow steps	Template Method	Batch jobs
Stateful logic	State	Order lifecycle
Centralized requests	Front Controller	DispatcherServlet
Pre/Post processing	Intercepting Filter	Auth, logging
Async processing	Command	Job execution
Messaging	Messaging Gateway	Kafka, JMS
Caching	Flyweight	In-memory caching

---

3 Used When Scale & Complexity Grow

Problem	Pattern	When to Use
Undo / rollback	Memento	Editors, transactions
Multiple handlers	Chain of Responsibility	Validation pipelines
Dynamic behavior	Decorator	Feature toggles
Lazy resource loading	Proxy	Hibernate lazy loading
Object reuse	Object Pool	DB connections
Distributed TX	Two Phase Commit	Banking systems

 Don't use these in small projects unless needed.

---

4 Rare but Important (Framework / Engine Level)

Pattern	Usage
Interpreter	Rule engines
Visitor	Compilers, analytics
Bridge	Cross-platform SDKs
Composite	Tree structures
Iterator	Custom collections

 You may not write these often, **but frameworks use them heavily**.

---

5 REAL MICROSERVICES STACK – PATTERN MAPPING

 API Layer

- **MVC**
- **DTO**
- **Front Controller**
- **Authorization Interceptor**

 Service Layer

- **Facade**
- **Strategy**
- **Template Method**
- **State**

 Data Layer

- **Repository**
- **DAO**
- **Unit of Work**
- **Identity Map**
- **Lazy Loading**

 Integration Layer

- **Messaging Gateway**
- **Service Locator**
- **Proxy**

 Infrastructure

- **Singleton**
- **Object Pool**
- **Thread Pool**
- **Async Method Invocation**

---

6 WHICH PATTERNS SHOULD YOU CHOOSE? (RULE OF THUMB)

Situation	Use This Pattern
Object creation depends on input	Factory
Need to switch logic dynamically	Strategy
Want to hide complexity	Façade
Cross-cutting concerns	Proxy / Decorator
Event-driven behavior	Observer
Stateful workflow	State
DB interaction	Repository
Async & messaging	Command / MQ

---

7 WHAT INTERVIEWERS REALLY EXPECT

They don't want definitions.
They want answers like:

"In our payment service, we used **Strategy** for payment types,
Factory to create payment handlers,
Facade for service abstraction,
Observer for event publishing,
and **Proxy** for security."

If you say this — **you sound like a tech lead**.

□ FINAL ADVICE (VERY IMPORTANT)

❌ Don't force patterns
❌ Don't use all patterns
✅ Use patterns **only to solve problems**

Good code uses fewer patterns, but the right ones.

If you want next:

- **□ "Which pattern for which interview question"**
- **□ End-to-end real microservice design using these patterns**
- **□ Common pattern misuse & mistakes**
- **□ How Spring internally uses these patterns**